

API Security and DSA Report Sections

Introduction to API Security

An API is a controlled boundary between a client and an application's data or services. API security protects that boundary so that only permitted clients can perform permitted actions on permitted data. It includes authentication, authorization, input validation, safe error handling, transport protection, and monitoring. These controls matter for this project because the API exposes financial SMS records and supports operations that read, create, modify, and delete transactions.

Authentication answers who is making a request; authorization answers what that authenticated identity may do. They are related but different controls. A robust API should also validate JSON payloads, reject malformed requests with predictable status codes, avoid leaking sensitive implementation details, and use HTTPS so credentials and transaction data are encrypted in transit. Authentication alone does not make an API secure if an attacker can intercept credentials, reuse them indefinitely, or access more records than the account should be allowed to access.

This project uses Python's `http.server` and protects every implemented endpoint with HTTP Basic Authentication. The API returns 401 Unauthorized when credentials are missing or invalid, validates the required transaction ID during creation, and returns JSON error responses for invalid input, missing records, and duplicate IDs. The transaction data is held in memory and is reloaded from `output.json` when the server starts, so persistence and production-grade audit logging are outside the scope of this prototype.

Why Basic Authentication Is Weak

Basic Authentication sends a username and password in every request, encoded with Base64. Base64 is an encoding rather than encryption, so anyone who can observe an unencrypted connection can recover the credentials. For that reason, Basic Authentication must be used over HTTPS in any real deployment. Even with HTTPS, the same long-lived password is repeatedly presented, which increases the impact of password reuse, credential leakage, phishing, and accidental exposure in logs or client configuration.

The current project uses a fixed development username and password in the server source. This is acceptable for demonstrating the authentication flow, but it is not suitable for production. A deployed service should store secrets outside source control, use strong password handling, rotate credentials, apply rate limiting, and grant each client only the permissions it needs. Basic Authentication also does not provide a built-in mechanism for short-lived access, delegated permissions, refresh tokens, or a clear identity and authorization model for multiple applications.

Stronger Alternatives

JSON Web Tokens (JWT)

JWT-based authentication issues a signed token after a user or client successfully authenticates. The client sends the token in an `Authorization: Bearer` header on later requests. A token can contain an expiry time, subject, issuer, and scopes or roles, allowing the API to validate claims without receiving the user's password on every request. Short expiration times and refresh-token rotation reduce the useful lifetime of a stolen access token.

JWT is useful for stateless APIs and distributed services, but it is not automatically secure. Tokens must be signed with a suitable algorithm, validated for issuer, audience, signature, and expiry, and transmitted over HTTPS. Sensitive data should not be placed in a JWT merely because its contents are encoded; the payload is normally readable by the bearer. Revocation and logout also require an explicit design because a stateless token may remain valid until it expires.

OAuth 2.0

OAuth 2.0 is an authorization framework for delegated access. It allows a client application to obtain limited access to a resource server without receiving the resource owner's password. Access tokens can be scoped, short-lived, and issued through flows suited to the client type. In a production MoMo integration, OAuth 2.0 would be a stronger fit when

separate applications or services need controlled access to transaction resources.

OAuth 2.0 is broader than simply replacing a password header and normally depends on a properly configured authorization server. The implementation must choose the correct grant flow, validate tokens and scopes, protect redirect-based flows where applicable, and use HTTPS. JWT can be the format of an OAuth access token, but OAuth 2.0 and JWT are not interchangeable concepts: OAuth defines an authorization protocol, while JWT defines a token format.

Comparison

Approach	Main benefit	Main weakness or responsibility	Appropriate use
Basic Authentication	Very simple to implement and test	Repeated long-lived credentials; requires HTTPS and careful secret management	Small internal prototypes and controlled demonstrations
JWT bearer tokens	Short-lived, claim-based, stateless API authentication	Revocation, key rotation, and strict claim validation must be designed	APIs serving authenticated users or services at scale
OAuth 2.0	Delegated access and fine-grained scopes	Requires an authorization server and correct flow configuration	Multiple clients, third-party access, and service integrations

For this coursework prototype, Basic Authentication makes the CRUD and 401 behavior easy to demonstrate. For a production financial API, HTTPS combined with an external identity provider, short-lived tokens, scopes, secret rotation, rate limiting, and audit logging would provide a substantially stronger security posture.

DSA Reflection

The parser processes 1,691 SMS messages into dictionaries with `transaction_id`, `transaction_type`, `amount_rwf`, `counterparty`, and the original body. The current run identified 8 authentication-code messages and excluded them from the transaction total. Of the remaining 1,683 messages, 1,676 matched a supported transaction pattern and 7 remained unpar sed. Messages without an ID are retained in the list, but they cannot be addressed by the API's ID-specific GET, PUT, or DELETE routes.

The comparison tested 20 IDs using two methods. Linear search scans the list from the beginning and compares each record's ID until it finds a match, so its worst-case time grows linearly with the number of records: $O(n)$. Dictionary lookup stores records under their IDs and uses the dictionary's hash table, giving expected constant-time lookup: $O(1)$. In the observed run, linear search times ranged from approximately 0.253 to 108.471 microseconds per lookup, while dictionary lookup ranged from approximately 0.060 to 0.148 microseconds. The exact timings vary with the machine and system load, but the dictionary was consistently faster, especially for IDs located later in the list.

The trade-off is memory and maintenance: the dictionary index uses additional memory and must be updated whenever a record is created or deleted. The API already maintains both structures for this reason: the list preserves the collection used by GET /transactions, while the dictionary makes ID-based operations efficient. For a substantially larger or persistent dataset, an indexed database column on `transaction_id` would be a stronger alternative because it provides efficient lookup without loading the entire collection into process memory and also supports durable storage. A balanced design would keep the index synchronized, enforce unique IDs, and measure performance with a larger and repeatable benchmark before choosing a production data structure.